Duke | THOMAS LORD DEPARTMENT of MECHANICAL ENGINEERING & MATERIALS SCIENCE

DRAFT Final Report - Autonomous Vehicle

Victor Xia, *victor.w.x@duke.edu*

***Abstract*—A report for the work done in semester Spring 2023 for course ME555 Automotive Engineering Design is provided. The goal is to start an effort for a vehicle to drive autonomously in a racing environment. A series of literature are briefly reviewed, and based on which a learning based control algorithm is selected and given effort to replicate. The algorithm is explained in detail, along with the current work progress and future work plans.**

I. Introduction

The field of autonomous vehicles has witnessed significant advancements in recent years, with numerous applications spanning from urban transportation to specialized environments such as motor sports. In the domain of racing, the challenges faced by autonomous vehicles include high-speed driving, aggressive maneuvering, and real-time decision-making. As a result, developing efficient control algorithms that can cater to the demanding requirements of racing scenarios is a critical aspect of success in autonomous car racing competitions.

The motivation for this project stems from our aspiration to participate in prestigious autonomous car racing competitions, including Indy Autonomous Challenge, Shell Eco-Marathon, and F1-Tenth. These events showcase cutting-edge advancements in the field of autonomous vehicles and provide an ideal platform for interdisciplinary teams to collaborate, innovate, and compete. The project starts with gaining a comprehensive understanding of the specific objectives and regulations governing these competitions.

A. Competitions

1) Indy: Indy Autonomous Challenge (IAC) is an autonomous vehicle racing competition held at the Indianapolis Motor Speedway. The primary objective of this event is to promote innovation and research in the field of autonomous vehicles, with a focus on high-speed racing. Teams are required to develop control algorithms that can guide their vehicles through a series of high-speed races, with speeds exceeding 200 mph. The competition emphasizes the importance of vehicle-to-vehicle communication, real-time decision-making, and robust path planning.

This competition is deemed very suitable for a long-term goal of this project due to its stringent performance requirements and high cost.

2) Shell: The Shell Eco-Marathon is an annual competition that challenges student teams from around the world to design, build, and test energy-efficient vehicles. The primary goal of this event is to encourage innovation in sustainable transportation and promote the development of energy-saving technologies. The central component of the competition focuses on energy efficiency, with teams striving to create vehicles that can travel the farthest using the least amount of energy.

3) F1/10: F1-Tenth is a highly competitive autonomous racing event that aims to bridge the gap between full-scale autonomous racing and scaled-down autonomous vehicle research. The competition challenges teams to develop and deploy control algorithms for 1/10th scale race cars, which must navigate a complex track autonomously at high speeds. The winning team is determined by evaluating the performance of their autonomous race car in terms of lap times, vehicle control, and the ability to navigate complex racing scenarios.

After discussion, the primary goal is set to develop an autonomous driving algorithm for the F1-Tenth competition. By tailoring our control algorithm to address the specific challenges and constraints of the F1-Tenth race, we aim to create a strong foundation for future participation in other competitions. This endeavor also allows us to acquire valuable skills and knowledge in the domain of autonomous vehicles and contribute to the development of safer and more efficient transportation solutions.

B. Related Works

There are numerous works and studies contributing to the development of autonomous racing vehicles. Reviewing them offers us valuable insights on cutting-edge methods to tackle the task.

[1] is a good starting point to overview the current market of the field. In the survey, the author categorize the works into three categories:

Global Planning Algorithms aiming to find an optimization of the path/trajectory to traverse through a given track or terrain.

Local Planning Algorithms focusing on maneuvers and locomotion policies to follow a given path/trajectory.

Behavioral Planning Algorithms giving the vehicle abilities to make decisions in certain situations and to perform in certain schemes to achieve complex tactical tasks (i.e. overtaking).

The author also described two trending methods to tackle these specific tasks,

Learning-based Commonly understood as Artificial Intelligence (AI). A series of machine learning, reinforcement learning and deep learning algorithms that adapts its control policies based on data and past experiences. They are usually end-to-end methods without a control hierarchy, but often requires enormous amount of training data and computational power.

Non-learning-based Usually achieved with control algorithms. These algorithms require a comprehensive understanding of the system and task at hand, needs more developing efforts, while providing far more safer and efficient results.

[2] proposes a novel algorithm to find a minimum curvature trajectory in a given race track geometry. Dedicated for the task of global planning, the ideal outcome would be finding a trajectory with minimum lap time. However, the solution of minimum time lap depends on physical properties and simulations to a degree of fidelity, requiring computational resources usually unacceptable for real-time computing. Many works thus retreat to finding a trajectory with minimum curvature, with some physical limitations under consideration and a slightly less ideal result. One might also see works seek to find the minimum distance path, optimizing calculation complexity. This over-simplified methods does not yield results good enough for racing tasks. The difference between a "path" and a "trajectory" is that a "path" is generated with out time being part of the consideration, while a "trajectory" has a series of time stamps associated with it dictating where a vehicle should be along the way at every moment.

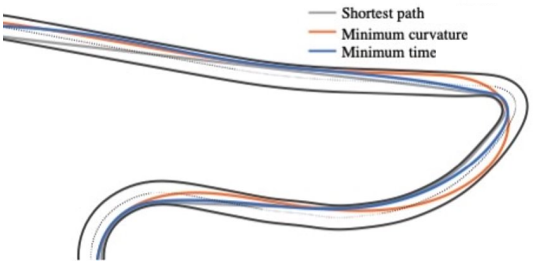

Figure 1: Three different paths often pursued in global planning problems: the shortest path, the minimum curvature trajectory, and the trajectory that would result in minimum time spent. Figure adapted from [1]

Specifically for F1-Tenth competition, the vast majority of teams and works use a hierarchical structure with both global planning and non-learning based local planning, as this is a naturally intuitive way of decomposing a complex mission. [3] uses an optimization algorithm to solve the global planning problem to find a trajectory from the vehicle's current position to a fixed horizon, then uses a Receding Horizon Model Predictive Control (MPC) to track the trajectory. The MPC is slightly adapted, with the control input being selected from a predetermined set of candidate maneuvers, to minimize the computation time. An obstacle avoidance algorithm is also briefly discussed in this paper.

[3] also uses a model commonly seen in works related to wheeled vehicle systems to predict and simulate the vehicle's behavior. The book [4] covers this physics of this model in detail and lays a very solid foundation for many researches and development works. The article [5] also propose a novel way to determine a model from the vehicle's travelling data, taking advantages of a linear Parameter Varying model whose instances are determined by a Pace Regression algorithm, to maintain fidelity even when the vehicle is operating near the boundary of its performance envelope while maintaining computational efficiency.

[6] proposed a coordination system tailored for describing geometries of the track and a vehicle's relative states to the track. This system is adopted by many researches including [3] and [7].

[7] proposed a brand new method for autonomous racing cars, combining the advantages of MPC and the idea of Q-learning from reinforcement learning. This novel approach share advantages with both learning and non-learning methods to address both global and local planning at the same time, with better performance than optimization based global planning, while ensuring safety along the race and a drastically improved training time. This paper is followed by thesis [8], which uses a slightly different setup and implemented physically on an F1-Tenth platform. The GitHub repository of the open-source code for [8], with ROS and various sensors integration, is published at https://github.com/mlab-upenn/LearningMPC.git.

Judging from the literature mentioned above, along with several other literature not documented, it is decided that the project shall begin by replicating and verifying the algorithm proposed by [7] and then seek ways to improve on top of this algorithm. By implementing the proposed control algorithm, we aim to demonstrate that our autonomous race car can successfully navigate the challenging racing environment while achieving high performance and adhering to the competition's requirements. If successful, the will-proposed improved controller could also result in a publish-worthy cutting-edge research.

II. Problem Setup

For the major part of the project, the primary goal is to realize an autonomous racing algorithm that can navigate a given race track efficiently, minimizing lap time while adhering to the constraints imposed by the track's geometry and vehicle dynamics.

A. Environment and Coordination System

The coordination system used for this project is taken from [6] as advised by [7]. The track is provided (measured physically or nominal for simulation) a series of anchor points with a Bezier curve, along with collection of its information such as center-line trajectory sample points in xyz coordinates in $< \vec{i}, \vec{j}, \vec{k} >$ base, denoted by $\mathbf{R}$. The center-line geometry is represented by $\mathbf{C}$. Along with it is the track width b, and a series of pre-processed information such as local curvature. The track is assumed to be straight before and after the finish line.

For a car driving on the track, a new type of local coordination is used, denoted by $\mathbf{T}$. This coordination system has the origin $P(s)$ on the center-line, corresponding the current progress of the vehicle along C. s denotes the distance traveled along the centerline, which also known as the curvilinear abscissa. The positive direction of the first axis of $\mathbf{T}$, $\vec{m}$, is along the local positive tangential direction of $\mathbf{C}$. The second axis, $\vec{n}$, is pointing to the left normal direction of local $\mathbf{C}$. The third axis is omitted but adheres to the right-hand rule. The illustration is drawn below in Fig. 2. This gives one a way to describe the current position of the vehicle in $\mathbf{T}$, denoted by M.

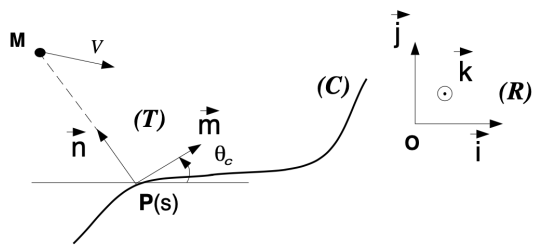

Figure 2: Illustrations of the two coordination systems, $\mathbf{R}$ is used to provide $\mathbf{C}$ geometry, based on which $\mathbf{T}$ is defined every moment as the car goes. M represent the vehicle's local position, v and θ_e represent the vehicle's velocity vector and local heading direction. This figure is taken from [6]

B. Vehicle States and Inputs

The vehicle's behavior is represented by a state vector $x \in \mathbb{R}^6$ includes the following element:

$$u = [v_x,\ v_y,\ w,\ e_a,\ s,\ e_d]^T \tag{1}$$

State Variable	Definition
v_x	Longitudinal velocity of the vehicle
v_y	Lateral velocity of the vehicle
w	Yaw rate
e_a	Difference between heading and tangent (same as θ_e)
s	Progress along the track centerline
e_d	Normal distance from the track centerline along $\vec{n}$

Table I: State Variables and Their Meanings

Note that the first three element of x, $x_{(1:3)}$, is in a sense of 1st-person-view state describing its motions, while $x_{(4:6)}$ describes its positions from 3rd-person-view.

The input vector for the vehicle system is $u \in \mathbb{R}^2$ includes:

$$u = [a,\ \delta]^T \tag{2}$$

Input Variable	Definition
a	Acceleration
δ	Wheel Steering Angle

Table II: Input Variables

The vehicle system is an arbitrary mathematical model $x^+ = f(x, u)$, which takes in the current state and input then output the new state at the next time sample. This discrete model is arbitrary and can be any nature for simulation purposes, and would be the actual physical vehicle and on-board sensors when testing. An implicit requirement of this model is to maintain a satisfying fidelity while keep the computational complexity low enough for real-time computing.

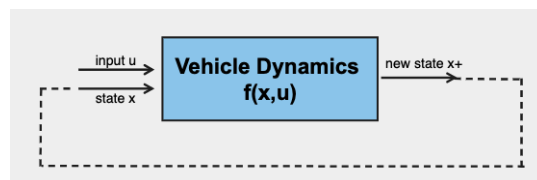

Figure 3: Block diagram showing the arbitrary model of the vehicle's dynamics, and how it takes in state and inputs, then outputs and iterates on new state.

C. Goal and Constraints

The high level problem can then be concluded as an optimization problem, whose objective is to minimize lap time while adhering to track constraints and ensuring vehicle stability. This is achieved by defining a suitable cost function J that captures these objectives, as well as incorporating both hard and soft constraints to represent the various limitations imposed by the track's geometry, vehicle dynamics, and competition rules. The constraints include, but are not limited to:

- The vehicle must not deviate from the track boundaries, defined by the left and right boundaries.
- The vehicle must maintain stability, avoiding situations that may lead to loss of control or overturning.
- The vehicle's control inputs must be within the physical limitations.

The resulting optimization problem will be solved using suitable numerical methods, and the developed algorithm will be tested and validated using simulation and, ultimately, real-world experiments.

D. Nomenclature

Symbol	Definition
$\mathbf{C}$	Track centerline
b	Track width
$x,\ X$	Vehicle state vector, and a series of state vectors
$u,\ U$	Vehicle input vector, and a series of input vectors
$t,\ t+1,\ T$	Current time, time at next time sample, and a series of time samples
$f,\ \{A,\ B,\ C\}$	The vehicle's model, and the linearized state space
Γ	The linearization regressor vector
j	The current number of laps
J	The cost function
N	Prediction Horizon
R	The input regulator, defines the cost of each input element
h	The time cost
Q	Same as in Q-learning, the learned estimated cost for future
z	The predicted terminal state at the end of the prediction horizon
$K,\ P$	Numbers of neighbors for K-nearest neighbor
κ	Kernel used to define "near" in K-nearest neighbor
$D(z)$	Set of the K-nearest neighbor in the previous 2 laps of the current predicted terminal state.
$S(z)$	The set of the states one-step forward of $D(z)$.
λ	A linear combination term, defined as a row vector with its elements being all non-negative and sum to 1
$*_{(i)},\ *_{(i:j)}$	The i-th element, or the i-th to j-th elements of the variable
$*_F$	The variable within the physically feasible region
$*_t$	The variable at time t
$*^+$	A new iteration of the variable
$*_{t+i\|t}$	The variable's i-th iteration predicted at time t
$*_T$	The variable at the end of the lap (the Terminal Set)
$\tilde{*}$	Arbitrary variable

Table III: Major variable and constants used for this project, along with their explanations.

III. Optimization

After careful consideration, the Learning Model Predictive Control (LMPC) algorithm proposed by [7] is selected to give effort to replicate and implement, for its numerous advantages and abundance of referencing literature and open-source codes.

The algorithm uses a classic Model Predictive Control (MPC) structure, while novelly includes a Q value in the cost function. Q is usually used in Reinforcement Learning (RL) as a method known as Q-learning, which aims to estimate the optimal action for a given state by quantifying the expected long-term reward for each state-action pair, and enabling the agent to make informed decisions based on previous histories.

This addition grants the MPC the learning ability, making it possible for the vehicle to improve its performance as it repeats driving itself on the track. This method also drastically shortens the training time, which is usually a major concern for most RL projects, thanks to the nature of MPC having known physical dynamics of the agent (the vehicle).

The algorithm solves the optimization problem setup as below, with a few adaptations made to better suit this project.

At each step, solve

$$J_{t\to t+N}(x_t,\ z_t) = \min_{U_t,\lambda_t} \sum_{k=t}^{t+N-1} [h(x_k) + u_k^T R u_k + s_k^T q s_k] + Q(z_t)\lambda_k \tag{3}$$

$$s.t.\quad x_k \in X_F,\ u_k \in U_F \tag{4a}$$
$$x_{k+1} = f(x_k, u_k) \tag{4b}$$
$$\lambda_t \geq 0, \mathbb{1}\lambda_t = 1, D(z_t)\lambda_t = x_{t+N} \tag{4c}$$
$$\forall k = t, t+1, ..., t+N-1 \tag{4d}$$
$$s_k \geq 0 \tag{4e}$$

Then do

$$u_t = U_t(1) \tag{5a}$$
$$x_{t+1} = f(x_t, u_t) \tag{5b}$$
$$z_{t+1} = S(z_t)\lambda_t \tag{5c}$$

The optimization problem is initialize at every "moment" while the car is racing in the track, and predict the vehicle's action, behavior, and race outcome within a fixed amount of time from the present to the future. Due to the discrete nature of on-board computing devices, every states and inputs are observed and given at instances of time. The input would then be hold as constant for a short duration of time, known as zero-order hold (ZOH), while the system state evolves continuously. This means the problem is discretized as "steps", where each step is where the vehicle observe its current state and makes its actions. The computation complexity must also be simplified enough be able to give calculation results before the next step arrives.

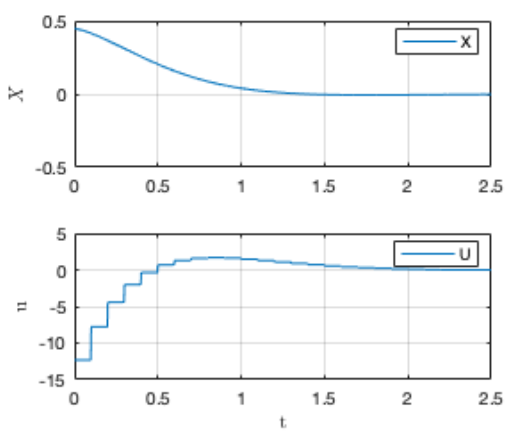

Figure 4: The nature of computer causing a series of discrete inputs U being acted upon a continuous real-world system X.

The above optimization problem is broken down and explained as follows:

A. Cost Function

1) Time Cost: $h(x_k)$ is the time cost of every step. It is a constant that adds up as more steps are taken to reach the

finish line, and ignores any states come after the finish line.

$$h(\tilde{x}) = \begin{cases} 1 & \text{if } \tilde{x} \notin X_T \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

This term encourages the solver of the optimization problem to minimize the steps needed, which in turn minimizes the total lap time.

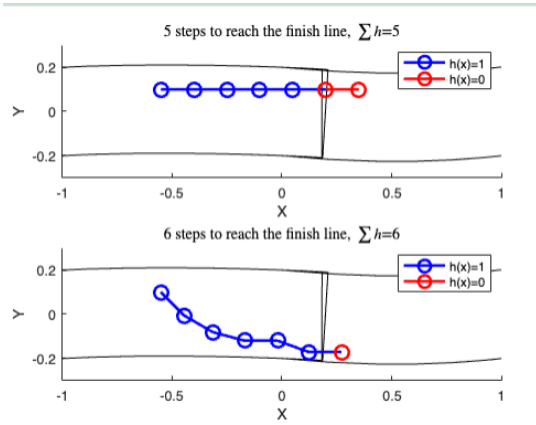

Figure 5: Showcase of the effect of $h(x)$ term. At x_t, the solver estimate the next 6 steps (N=6). The trajectory above takes 5 steps to reach the finish line ($\sum h = 5$), and the below one takes 6 steps ($\sum h = 6$). The solver is encouraged to find the input sequence that result in the minimum $\sum h$.

2) Quadratic Regulator: $u_k^T R u_k$ is the quadratic regulator term to minimize the input value. R is a diagonal matrix with each element on its diagonal weighing the cost of each input element. For an F1-Tenth racing car, acceleration is considered as "cheap" and encouraged, while steering is "expensive" and should be conservative. Based on the input elements' magnitudes, the $R_acceleration$ term should be even smaller allowing large acceleration values. Note that $u_k^T R u_k$ is the matrix form as same as $R * u^2$.

Nominal value:

$$R = diag([0.1\ 1]) \tag{7}$$

3) Slack Variable: $s_k^T q s_k$ is the slack variable, with q being the quadratic weight. This is to be used with soft constraints, which are to be explained later on. It allows the vehicle to break certain constraints 4e in case there is no feasible solution for the optimization problem.

4) Learning Term: $Q(z_t)\lambda_k$ is the learning term, representing the minimum time it would cost in the future to drive from the current state to the finishline. This knowledge is collected and learnt at the end of each lap, and the memory would be kept for two laps. More precisely, the algorithm would record its states while the car is driving, and review these states when it hit the finishline. The time spent on each lap would be queried at the finishline, and the trajectory would be traced back and the past states would be updated with the time-to-go $Q(\hat{x})$.

Time is treated as priority for F1-Tenth racing, so it is intuitive to use this time-to-go directly as the Q-value. Note that in classic RL, the agent seeks actions that provide the maximum Q, whereas in this LMPC algorithm, the actions with the minimum Q is the target. To acquire the optimal action, the assumption is made that Q is linear within the 6D state space (where $x \in \mathbb{R}^6$ is in). This means a solution can be found by a linear combination term λ combining the memorized states near the car's current state.

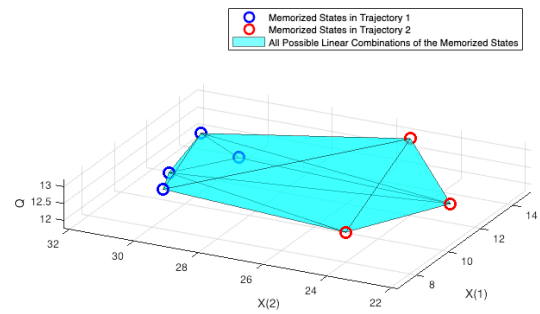

Figure 6: The region of all possible linear combinations of the nearest states. The figure is showing a 7-nearest neighbors with 4 from the one of the previous laps and 3 from the another, with the Q value and the first two elements of the state x. Note that this region defined by all possible λ is just the convex hull of the neatest states.

This linear combination term of K-nearest neighbors is used on z_t, the candidate terminal state at time step t. z represents the state that the system predicts the vehicle would arrive at the end the prediction horizon $t+N|t$. This value is hold constant every time the optimization problem is being solved, and updated after the vehicle has made its action.

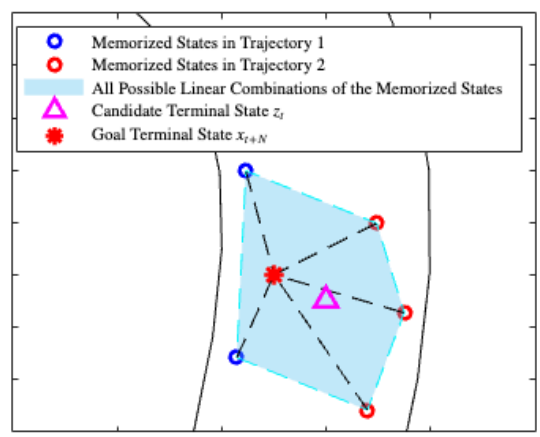

Figure 7: At given time t and z_t, the K-nearest memorized states of z_t is found. These memorized states is the set $D(z_t)$, and within their convex hull, a goal terminal state with the minimum interpolated Q, x_{t+N}, is computed. The system then tries to drive the vehicle towards x_{t+N}.

While z is updated independently, the vehicle does not try to drive towards it. Instead, a goal terminal state x_{t+N} is computed using $D(z_t)$, and the vehicle will solve the optimization problem 3 trying to find a series of $U = \{u_t,\ u_{t+1},\ ...,\ u_{t+N}\}$ that result in the vehicle drive to x_{t+N} in N steps as accurate as possible. Finally the car will execute the first entry of U as input, and re-initialize the problem with the next step's observations. This procedure is common standard MPC mechanism.

It is worth to mention that though the states are 6-dimensional, not all elements are needed or weigh the same in finding the meaningful neighbors. So instead of

using Euclidean or Manhattan norm, one could customize one's own distance kernel matrix κ and define the distance $||\Delta x||_\kappa^2 = \Delta x^T \kappa^T \kappa \Delta x$

The high-level goal of this term is to keep the problem convex. A convex problem means that the problem has only one optimum, and the computation complexity is usually more acceptable. If one could find a reliable and efficient non-convex solver, then this term could be replaced with some thing more intuitive.

B. Constraints

1) 4a: $x_k \in X_F,\ u_k \in U_F$ is to say that all states and inputs at all times must be physically feasible. This could include motor power output limitations, steering mechanism limitations, tire grip limitations, etc. One could also put in entries for safety considerations and competition rules. The controller structure means that, compare to classic AIs, these constraints are strictly complied and safety is thus enforced.

2) 4b: $x_{k+1} = f(x_k, u_k)$ is the vehicle's dynamics–what the vehicle would do when it is at its current state, with the input received. The output is the new state, which is recursively used to predict the further steps forward.

This model can be non-linear, with all physical aspects under consideration, or linearized around a suitable pivot state. Euler's method states that around a pivot state $\tilde{x}$, there exists

$$\tilde{x}^+ \approx \tilde{x} + \Delta t * \nabla f(\tilde{x}) \tag{8}$$

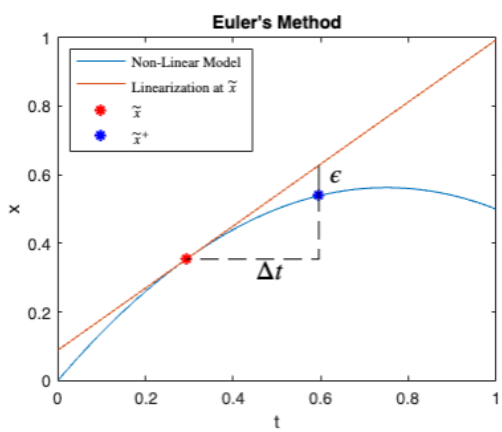

Figure 8: Illustration of Euler's method. The non-linear model is linearized at $\tilde{x}$ and $\tilde{x}^+$ can be predicted. If Δt is small enough, the error ϵ between the linear prediction and the real $\tilde{x}^+$ would be small enough and negligible.

This is to say that the gradient of the model at the pivot state can be used to transform the model into a linear model, with a form of $x_{k+1} = A * x_k + B * u_k$ and more efficient computation. The drawback, however, is that the linear model will lose fidelity when the current state moves far from pivot state. This is especially true for a vehicle operating near the boundaries of its performance envelope, where dynamics becomes extremely non-linear. This will be discussed further at Section IV.

3) 4c: $\lambda_t \geq 0, \mathbb{1}\lambda_t = 1, D(z_t)\lambda_t = x_{t+N}$ is the linear combination term mentioned at above $Q(z_t)\lambda_k$ discussion. To achieve this, $\lambda \in \mathbb{R}^{2K}$ shall be a vector with all of its elements being non-negative, and the Manhattan Norm of the vector being 1 ($\mathbb{1}$ means a row vector of 1). Fig 7 shows the usage of λ.

4) 4d: $\forall k = t, t+1, ..., t+N-1$ is the mathematical expression of saying, in a discrete time fashion, the solver iterate from the current time t to a fixed horizon of future $t+N$. The prediction is made from a clean slate at every time sample t.

5) 4e: $s \geq 0$ is the slack variable of soft constraints. Though soft constraints are not used in the project for now, the entry is still kept in case it is needed in the future. This term has proved to be useful in many other researches.

The general idea of soft constraint is that, for example, instead of having a hard constraint form such as $x_k < x_{max}$, the constraint is measured by the amount of its violation $s = max\{x_k - x_{max},\ 0\}$. When this is multiplied by a quadratic weight $s_k^T q s_k$, the solver is encouraged to minimize the violation, if possible. In extreme cases where constraint violation is inevitable, the solver would not dwell on finding a perfect solution that does not exist. This could be used for constraints such as safe speed limit or reversing longitudinal speed.

C. Actions

These below are the actions the system takes after the optimization problem 3 is solved.

1) 5a: $u_t = U_t(1)$ means that, as mentioned briefly above, the vehicle solves for a series of inputs but only carry out the first entry. The optimization problem 3 is then re-initialized after the input action is taken.

It is also possible to carry out multiple inputs before re-initialize the optimization. This would save computational intensity at the expense of control performance.

2) 5b: $x_{t+1} = f(x_t, u_t)$ is the same as 4b. The difference of the meanings behind them is that 4b is the simulation model that happens in the computer calculation prediction process, where as 5b refers to the physical evolution of the car in the real world. This is to describe what happens when the car carries out the input u_t.

3) 5c: $z_{t+1} = S(z_t)\lambda_t$ is the independent update scheme of the candidate terminal state. As mentioned above, $D(z_t)$ is the set of K-nearest neighbors of z_t. Then $S(z_t)$ describes the states that are one step forward of those in $D(z_t)$. The significance of this action is to update z based on how the past trajectories evolves. When each lap is finished and new experience learnt, z is reinitialized using the N-th step of the last lap.

IV. Vehicle Dynamics

With the optimization scheme discussed, one of the most important aspect for nearly all controller design raises: a mathematical model is required to describe the vehicle's motion. There are many ways to get this $x+ = f(x, u)$, two of them are briefly described below.

The first one is used and discussed in detail by [7], which is a two-parts linearization procedure aims to find every $x_{k+1} = f(x_k, u_k) \approx A_k x_k + B_k u_k + C_k$ in the optimization problem 3.

A. Dynamics

The first part of the procedure describes $x_{(4:6)}$, which is the 3rd-person-view of the vehicle's position and progress on the track.

$$\dot{s} = \frac{v_x \cos(e_a) - v_y \sin(e_a)}{1 - Curv(s) * e_d} \tag{9a}$$

$$\dot{e_a} = w - \dot{s} * Curv(s) \tag{9b}$$

$$\dot{e_d} = v_x \sin(e_a) + v_y \cos(e_a) \tag{9c}$$

where $Curv(s)$ is the local curvature of the track centerline geometry at progress s.

This can be linearized using Euler's Method 8 and become

$$x_{t+1,(4:6)} = \begin{bmatrix} e_{a,t+1} \\ s_{t+1} \\ e_{d,t+1} \end{bmatrix} \tag{10}$$

Note taht these values can be regarded as functions of

B. Kinematics

The second part of the procedure describes $x_{(1:3)}$, which is the 1st-person-view of the vehicle's motion and angular motion. As this part is heavily dependent on the forces acting upon the car, plus the effects of elasticity, viscosity and friction, it would not be wise to dive deep into mimicking how the world actually works. Instead, simple machine learning (ML) algorithm such as regression usually works well enough in capturing the statistical pattern, while ignoring the unnecessary details.

A regression vector is proposed

$$\Gamma^l(\tilde{x}) = arg \min_{\Gamma} \sum_{i \in D_P(\tilde{x})} \kappa_{Epa}\left(\frac{||\tilde{x} - x_i||^2_{\kappa_P}}{\texttt{bandwidth}}\right) y_i^l(\Gamma) \tag{11}$$

where

- $D_P(\tilde{x})$ is the P-nearest neighbor of $\tilde{x}$. Generally speaking, P should be greater than K since P is used for training the regression.
- κ_{Epa} is the Epanechnikov Kernel function:

$$\kappa_{Epa}(\tilde{\xi}) = \begin{cases} \frac{3}{4}(1 - \tilde{\xi}^2) & \texttt{if } |\tilde{\xi}| < 1 \\ 0 & \texttt{otherwise} \end{cases} \tag{12}$$

- `bandwidth` is a user defined value.
- κ_P is the distance kernel similar to κ used in the previous section. One could set it to be different than κ should desired.
- $l = \{v_x,\ v_y,\ w\}$ and

$$y_i^{v_x}(\tilde{\Gamma}) = ||v_{x,i+1} - \tilde{\Gamma}[v_{x,i},\ v_{y,i},\ w_i,\ a_i,\ 1]^T|| \tag{13a}$$

$$y_i^{v_y}(\tilde{\Gamma}) = ||v_{y,i+1} - \tilde{\Gamma}[v_{x,i},\ v_{y,i},\ w_i,\ \delta_i,\ 1]^T|| \tag{13b}$$

$$y_i^{w}(\tilde{\Gamma}) = ||w_{i+1} - \tilde{\Gamma}[v_{x,i},\ v_{y,i},\ w_i,\ \delta_i,\ 1]^T|| \tag{13c}$$

with $\Gamma \in \mathbb{R}^5$ being a row vector.

This is to say that the regression model is solving for optimal values of 3 vectors $\Gamma^{v_x}(x_k),\ \Gamma^{v_y}(x_k),\ \Gamma^{w}(x_k)$ for every x_k in the optimization problem 3. These vectors, with simple linear operation with the state x_k and input u_k, will give the next state x_{k+1} just like the complicated physical computation would do. The linear operation is as follows:

$$\begin{aligned} x_{k+1,(1:3)} = & \begin{bmatrix} \Gamma^{v_x}_{(1:3)}(x_k) \\ \Gamma^{v_y}_{(1:3)}(x_k) \\ \Gamma^{w}_{(1:3)}(x_k) \end{bmatrix} x_{k,(1:3)} \\ & + \begin{bmatrix} \Gamma^{v_x}_{(4)}(x_k) & 0 \\ 0 & \Gamma^{v_y}_{(4)}(x_k) \\ 0 & \Gamma^{w}_{(4)}(x_k) \end{bmatrix} u_k \\ & + \begin{bmatrix} \Gamma^{v_x}_{(5)}(x_k) \\ \Gamma^{v_y}_{(5)}(x_k) \\ \Gamma^{w}_{(5)}(x_k) \end{bmatrix} \end{aligned} \tag{14}$$

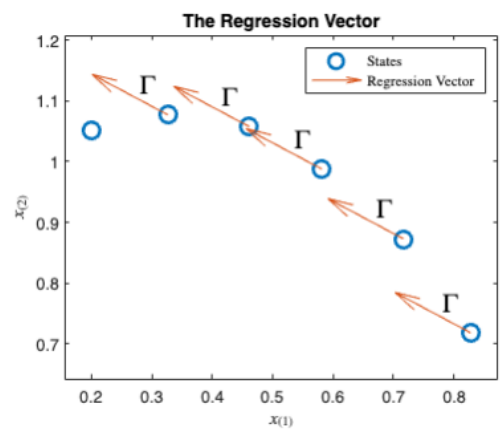

Figure 9: The concept of regression vector. Learnt from the collection of states, this vector simplifies the states' non-linear noisy evolution into a linear transformation represented by Γ. Though this inevitably results in prediction errors, it would be acceptable it is only used near the center region of the collection with large P and small Δt.

C. Affine Time Varying Model

With the procedures in IV-A and IV-B, one can define an affine time varying (ATV) model.

$$x_{k+1} = A_k x_k + B_k u_k + C_k \tag{15}$$

With the time varying matrices

$$A_k = \begin{bmatrix} \Gamma^{v_x}_{(1:3)}(x_k) & 0 & 0 & 0 \\ \Gamma^{v_y}_{(1:3)}(x_k) & 0 & 0 & 0 \\ \Gamma^{w}_{(1:3)}(x_k) & 0 & 0 & 0 \\ (\nabla_x e_{a,k+1}(x_k))^T \\ (\nabla_x s_{k+1}(x_k))^T \\ (\nabla_x e_{d,k+1}(x_k))^T \end{bmatrix} \tag{16a}$$

$$B_k = \begin{bmatrix} \Gamma^{v_x}_{(4)}(x_k) & 0 \\ 0 & \Gamma^{v_y}_{(4)}(x_k) \\ 0 & \Gamma^{w}_{(4)}(x_k) \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \tag{16b}$$

$$C_k = \begin{bmatrix} \Gamma^{v_x}_{(5)}(x_k) \\ \Gamma^{v_y}_{(5)}(x_k) \\ \Gamma^{w}_{(5)}(x_k) \\ e_{a,k+1}(x_k) - (\nabla_x e_{a,k+1}(x_k))^T x_k \\ s_{k+1}(x_k) - (\nabla_x s_{k+1}(x_k))^T x_k \\ e_{d,k+1}(x_k) - (\nabla_x e_{d,k+1}(x_k))^T x_k \end{bmatrix} \tag{16c}$$

for $\forall k = \{t, t+1, ..., t+N-1\}$.

D. Data-driven Linear Time Parameter Model

Fényes and their colleagues in their work [5] proposed a new novel data-driven method of acquiring a linear parameter varying (LPV) model for the vehicle.

Different than ATV system described above, the LPV model does not try to determine the current model as time progresses. Instead, many set of parameters for the model is determined at the same time, each corresponds to a region of states. And when operating, the current state of the vehicle is identified and categorized into one of the regions, in which the parameters are assumed to be constant.

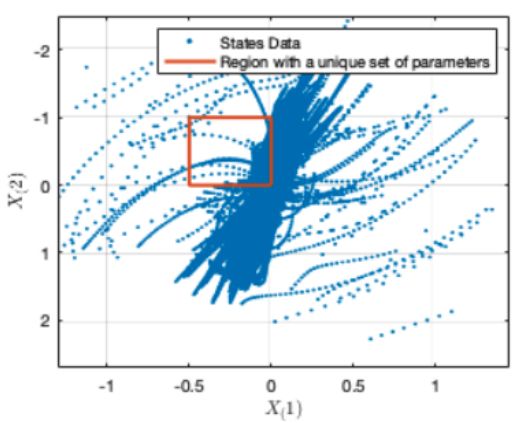

Figure 10: Concept of LPV models. States data are measured from experiments and categorized into regions. All states in one region, as contained in the red rectangle, share the same parameters. If the state evolves into a new region, a new set of parameters would be activated. Image taken and adapted from [5].

Benefiting from the amount of data used for building the model before any races, the car commence races with more knowledge of its own nature. This allows the car to maneuver more aggressively, knowing fully well how to manage its states, especially at extreme states. Due to this, it is recommended that the project to switch to this modeling method in the future. As for now, the project is using ATV model in IV-C.

V. Current Development

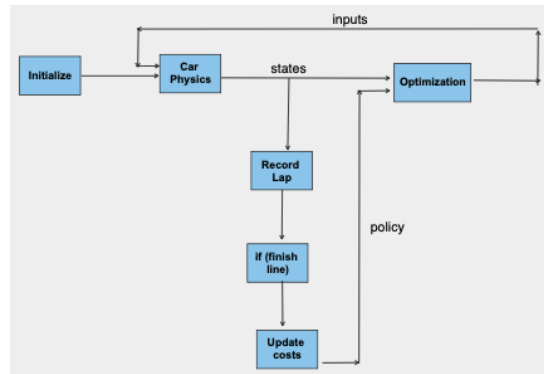

Figure 11: Flow chart of the project development. The upper part of the "T" shape is the controller part, and lower part is the learning part.

Up until this report is written, the coding work of the first version of the algorithm code in MATLAB is concluded. However, the debugging requires more effort than anticipated. Due to the workload, labor, and schedule of the semester, the project requires further development work. (see SectionVI) All fruit of the project so far is stored at Duke's GitLab service at https://gitlab.oit.duke.edu/wx75/lmpc-autonomous-vehicle.git

The most challenging part of the development work is to realize the optimization process. This process is responsible for solving the optimization problem using appropriate solver and formulation format. Several different methods were explored during the development, which are discussed below.

A. Offline Computing - Dynamic programming

One kind of methods are offline computing. Offline computing conducts the bulk of the simulating, regression and optimization computations before the vehicle starts moving. For this project specifically, the computation would be done at the start of each lap (at finishline), on a high-performance computer. The computation would generate a set policies for all possible states the vehicle might encounter in the following lap, then upload the policy to the vehicle. The vehicle only need to query the policy of the best action of the current state, and carry out the policy's feedback.

A common offline computing method for MPCs is Dynamic programming (DP). DP estimates the cost hyper-grid-surface of every states that are 1-step before any terminal states, $J_{N-1\to N}(X_F)$. Using this result, it computes the cost of states 2-steps before the terminal states, to the terminal states, and goes on. This process propagates back all the way until it reaches the prediction horizon, as shown below:

$$
\begin{aligned}
& J_{N-1\to N}(X_F) \\
& J_{N-2\to N}(X_F) &=& \; J_{N-2\to N-1}(X_F) + J_{N-1\to N}(X_F) \\
& J_{N-3\to N}(X_F) &=& \; J_{N-3\to N-2}(X_F) + J_{N-2\to N}(X_F) \\
& \quad \ldots\ldots \\
& J_{1\to N}(X_F) &=& \; J_{1\to 2}(X_F) + J_{2\to N}(X_F)
\end{aligned}
$$

The result would be a hyper-surface of costs in the state-action space, which provides information of the cost of the next 10 steps for every possible state-action pair. When the car is operating, the system refers to the surface with the current state, interpolate and finds the action that would yield the minimum expected cost.(see Fig12)

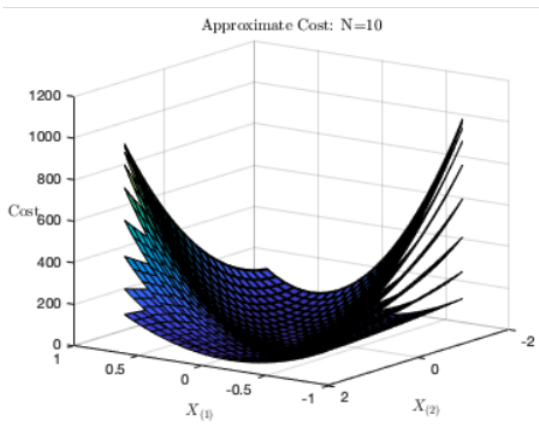

Figure 12: Policy grid generated by DP. There are a total of 10 grid surfaces in the policy space corresponding to a prediction horizon of $N = 10$. The bottom surface corresponds to the estimated cost of 1-step before terminal state, and the top one the cost of 10-steps before. When the vehicle is racing, it finds its current state in the grid and interpolate the cost on the most top surface to find the best (least expected cost) action for the future 10-steps.

B. Online Computing - Quadratic Programming

Online computing means that every calculation happens along with the system progress (car races). This requires a greater on-board computation resource, or high communication bandwidth for low latency off board computing. But the advantages of online computing is that the system performs well even when it enters unanticipated states, or environments of non-repetitive nature.

To do this, the common method is quadratic programming (QP), which reformat and simplifies the optimization problem 3 into the following form:

$$\min \; \frac{1}{2} x^T H x + q^T x \tag{17a}$$

$$s.t. \; \text{lowerbound} \leq A_c x \leq \text{upperbound} \tag{17b}$$

where x is the decision matrix, H is the Hessian matrix, q is the gradient matrix, A_c is the constraint matrix.

x includes everything that the problem is trying to solve: series of states, series of inputs, $lambda$, and series of s. The solver would try to solve the optimization problem 3 to find all of these at the same time. This technique is called the batch method. Due to the linear nature of the problem and the quadratic form, solving the problem could be computationally efficient enough for on-board real-time computing.

There are two solvers used for this project. `fmincon()` the MATLAB built-in optimization solver, and the YALMIP open-source tool package.

VI. Future Works

As the work is not concluded, the result of this project is yet to be shown. It is thus decided that the development shall be continued and the Git repository updated during the summer. The definition of completion is when the project reaches a degree that the it becomes suitable to serve for new research topics, or to be taken over and adapted for other projects (DEV, Duke Motorsports, etc.).

Other than solving the code bugs, the overall structure of the code could also use some clean up. The code shall be transformed to Simulink for better visualization, update iterations, and for tweaking the hyper parameters and weights to find the best performance values.

In long term, the algorithm could be implemented onto a physical F1-tenth car to conduct experiment or to participate in racing competitions. To do this, soft constraints must be included for safety concerns, and a localization method must also be implemented on the car for state identification.

It is also worth to mention that, per the advice of Dr. Bridgeman, this controller could be improved by re-configuring the optimization terms, and the result of the improvement could become a publish-worthy cutting-edge research. One would also like to consult the possibility of transforming such a research into a graduation thesis.

VII. Conclusion

The intention, work done, and the underlying theories of this project is presented in this report. The project aims to develop and autonomous driving algorithm for an F1-tenth vehicle in a track race environment. The related works are briefly reviewed, and a novel LMPC algorithm is explained and given effort to replicate. Due to multiple reasons the result of the deed is limited for the time being. The work shall continue during the summer break period, and the tentative plan is provided.

ACKNOWLEDGMENT

The author of this report would like to express gratitude to Dr. George Delagrammatikas for their engaging course ME555 Automotive Engineering Design that makes this project possible, and to Dr. Leila Bridgeman for their advice and guidance throughout the project.

The author would also like to convey appreciation to their esteemed colleagues and classmates Gabriela Morales, Garrett Smith, and Simona Tomar, for their highly professional scholarship in support of the success of this project.

REFERENCES

[1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022. [Online]. Available: https://doi.org/10.1109%2Fojits.2022.3181510

[2] S. Bari, A. S. Haidari, and D. Wollherr, "A fast approach to minimum curvature raceline planning via probabilistic inference," 2023.

[3] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale RC cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, jul 2014. [Online]. Available: https://doi.org/10.1002%2Foca.2123

[4] R. Rajamani, *Vehicle Dynamics and Control*, ser. Mechanical Engineering Series. Springer New York, NY, 2011.

[5] D. Fényes, B. Németh, and P. Gáspár, "A novel data-driven modeling and control design method for autonomous vehicles," *Energies*, vol. 14, no. 2, p. 517, Jan 2021. [Online]. Available: http://dx.doi.org/10.3390/en14020517

[6] A. Micaelli and C. Samson, "Trajectory tracking for two-steering-wheels mobile robots," *IFAC Proceedings Volumes*, vol. 27, no. 14, pp. 249–256, 1994, fourth IFAC Symposium on Robot Control, Capri, Italy, September 19-21, 1994. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1474667017473228

[7] U. Rosolia and F. Borrelli, "Learning how to autonomously race a car: A predictive control approach," *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2713–2719, 2020.

[8] R. M. Yuwei Wang, Baihong Zeng, "Learning model predictive control on f1tenth race car," University of Pennsylvania, Tech. Rep., 2020. [Online]. Available: https://drive.google.com/file/d/1dcdwietDdhKs-22mmAwxYzmE34k9dRWR/edit